

CHAPTER 17 — HANDS-ON PROJECTS

Deploying a Bastion Host in AWS using Terraform

A recap of real-world Terraform projects, then a secure jump-server architecture for private AWS resources.



Real-World Projects

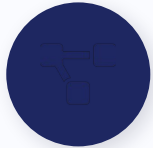


Bastion Architecture



Secure Access

What we'll instruct today



Part 1

Recap: Real-World Terraform Projects

Web app, Kubernetes, CI/CD automation, multi-cloud



Part 2

Bastion Host Overview

What a bastion host is and why it matters for VPC security



Part 3

Building the Architecture

VPC, subnets, security groups, bastion & private instance



Part 4

Connecting Securely

SSH hop from bastion into the private instance

PART 1 — RECAP

Real-World Terraform Projects

Four practical builds that put earlier chapters into production-style practice.

RECAP

Highly Available Web App in AWS



Auto Scaling ensures high availability. A separate database layer improves security and independent scaling.

RECAP

Kubernetes Cluster with Terraform (EKS)

```
module "eks" {  
  source      = "terraform-aws-modules/eks/aws"  
  cluster_name = "my-cluster"  
  cluster_version = "1.27"  
  subnets    = ["subnet-xxxxx"]  
  vpc_id      = "vpc-xxxxx"  
}
```

```
resource "kubernetes_deployment" "nginx" {  
  spec { replicas = 2 }  
}
```



EKS Cluster
v1.27

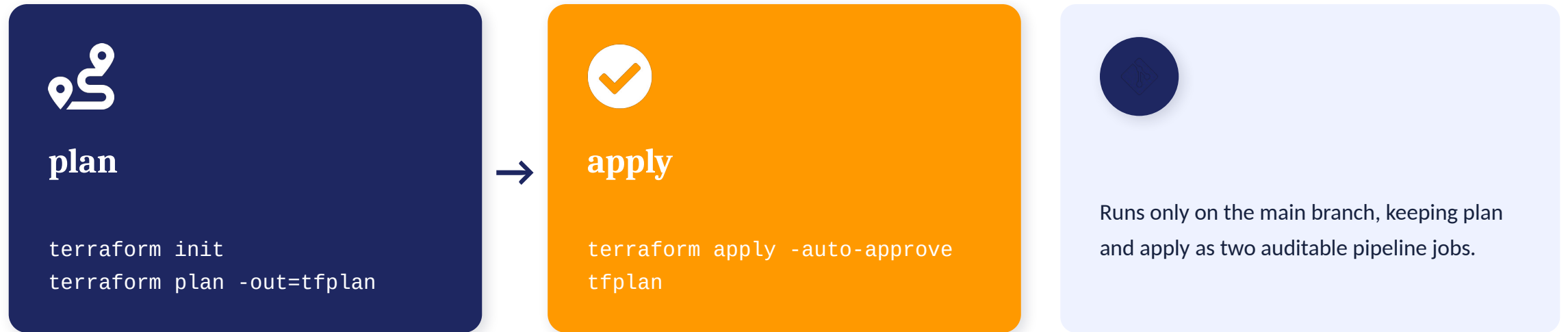


nginx Deployment

2 replicas • image: nginx:latest

RECAP

Automate Provisioning in a CI/CD Pipeline



GitLab CI/CD automates Terraform deployments end to end.

RECAP

Multi-Cloud Setup: AWS & Azure



Identical infrastructure across providers ensures redundancy and failover.

RECAP SUMMARY

Four Projects, One Toolchain



Scalable web app

Auto Scaling & Load Balancer for high availability



Kubernetes on AWS

Provisioned an EKS cluster and deployed Nginx



CI/CD automation

Terraform deployments driven by GitLab pipelines



Multi-cloud strategy

Matching infrastructure across AWS & Azure

PART 2 — DEEP DIVE

Deploying a Bastion Host in AWS

A secure jump server pattern for reaching private instances inside a VPC.

OVERVIEW

What Is a Bastion Host?



A Bastion Host is a secure jump server that provides access to private instances within a VPC.

Key Components



VPC

with public and private subnets



Security Groups

for controlled SSH access



Bastion Host

EC2 instance in the public subnet



Private EC2 Instance

accessible only via the Bastion Host

STEP 1

BASTION HOST

Define the AWS Provider



```
provider "aws" {  
  region = "us-east-1"  
}
```



Every resource in this project is provisioned in the us-east-1 region
— keep it consistent across all steps.

STEP 2

BASTION HOST

Create a VPC & Subnets

VPC — 10.0.0.0/16



Public Subnet

10.0.1.0/24

`map_public_ip_on_launch = true`



*Bastion Host
(next step)*



Private Subnet

10.0.2.0/24

no public IP



*Private Instance
(later step)*

Why two subnets?

- ✓ Public subnet routes to the internet — only the bastion lives here
- ✓ Private subnet has no direct internet route — real workloads stay hidden
- ✓ Traffic between them is controlled entirely by security groups

STEP 3

BASTION HOST

Security Group for the Bastion Host

```
resource "aws_security_group" "bastion_sg" {
  vpc_id = aws_vpc.main_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["YOUR_IP/32"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```



Ingress locked to YOUR_IP/32 — only your machine can SSH into the bastion on port 22.



Egress is open (-1 / 0.0.0.0/0) so the bastion can reach the private instance and package repos.

STEP 4

BASTION HOST

Deploy the Bastion Host

```
resource "aws_instance" "bastion" {  
  ami          = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  subnet_id    = aws_subnet.public_subnet.id  
  security_groups = [aws_security_group.bastion_sg.name]  
  
  tags = {  
    Name = "BastionHost"  
  }  
}
```



BastionHost

t2.micro • public subnet

Reachable only via SSH (22) from your IP, per the bastion security group.

BASTION HOST

Private EC2 Instance (Bastion-Only Access)

```
resource "aws_security_group" "private_sg" {  
  vpc_id = aws_vpc.main_vpc.id  
  ingress {  
    from_port   = 22  
    to_port     = 22  
    protocol    = "tcp"  
    security_groups = [aws_security_group.bastion_sg.id]  
  }  
}  
  
resource "aws_instance" "private_instance" {  
  subnet_id      = aws_subnet.private_subnet.id  
  security_groups = [aws_security_group.private_sg.name]  
}
```



The one rule that matters

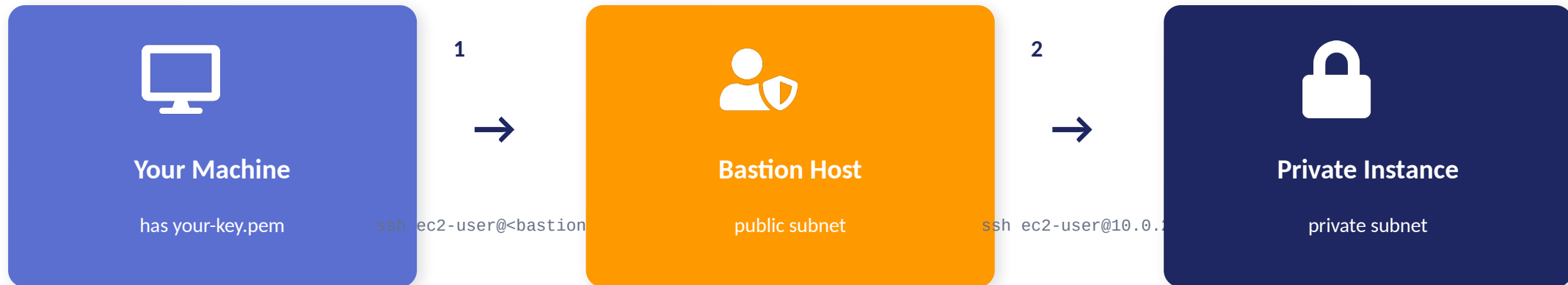
The private security group's ingress source is not an IP — it's the bastion's security group itself.

That means SSH traffic is only permitted if it originates from an instance wearing the `bastion_sg` — no other path in.

STEP 6

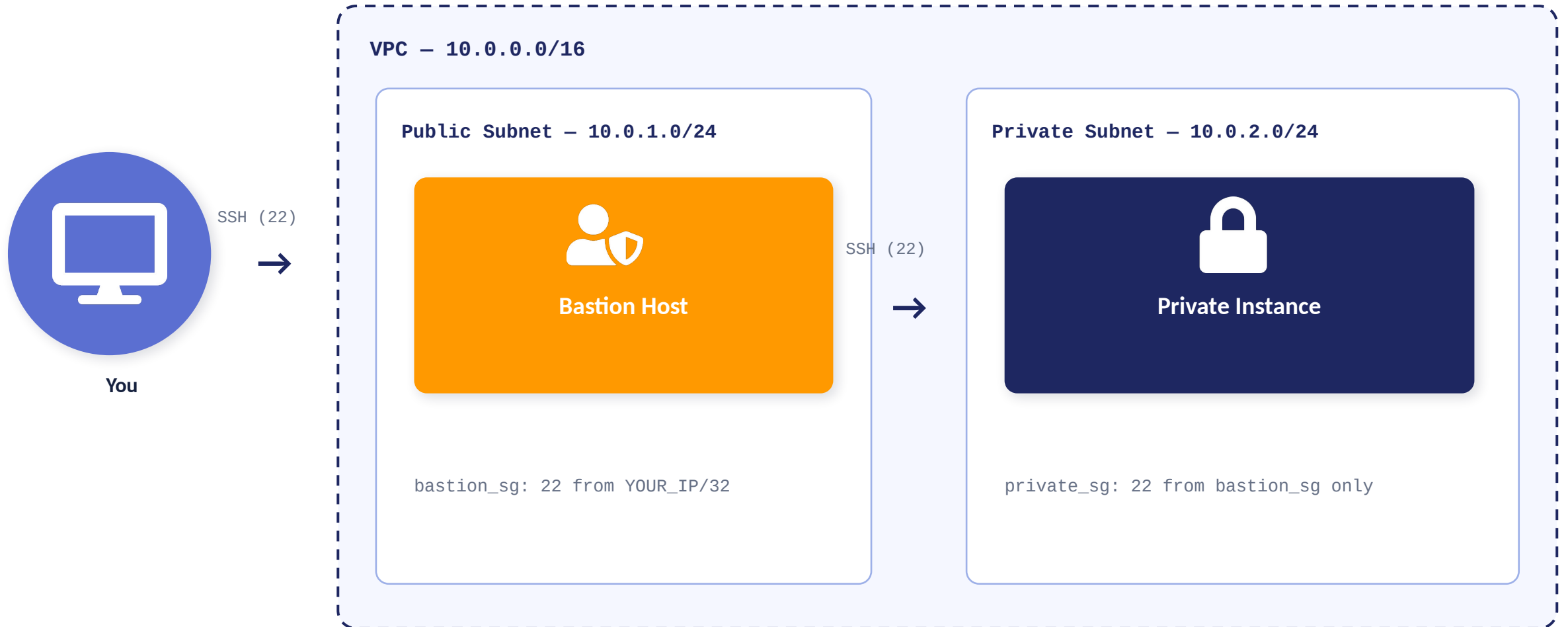
BASTION HOST

Connect to the Private Instance via Bastion



- 1 SSH into the Bastion Host from your machine using its public IP.
- 2 From inside the Bastion, SSH into the Private Instance using its internal IP.

Putting It All Together



CHAPTER SUMMARY

Key Takeaways



Secure jump server

Created a Bastion Host in the public subnet as the only entry point.



Locked-down ingress

Security group restricts SSH to a single known IP address.



Isolated workloads

Deployed a private EC2 instance with no direct internet exposure.



Bastion-only path in

Private instance accepts SSH only from the bastion's security group.